EXTREME SECURITY & UI AUDIT · V1.4.0

slop-deps

AI Dependency Hallucination & Supply-Chain Sentinel

A full-codebase review of the zero-dependency scanner that catches AI-hallucinated and typosquatted packages across npm, PyPI and crates.io. This audit combines static code review, live exploitation tests, browser-rendered UI verification and dependency hygiene analysis across all 78 files of the project.

3	5	6	3
HIGH	MEDIUM	LOW	INFO

Auditor: Z.ai Automated Security Review

Scope: CLI core · dashboard server · marketing site · CI pipeline · docs

Date: 3 November 2026

Table of Contents

1. Executive Summary 2

2. Audit Scope and Methodology 3

3. Findings Overview 4

4. High-Severity Findings 5

5. Medium-Severity Findings 7

6. Low-Severity and Informational Findings 8

7. Security Controls Verified as Working 9

8. UI / UX Review 10

9. Dependency and Supply-Chain Hygiene 11

10. Privacy Claims Verification 12

11. Priority Fix Roadmap 13

1. Executive Summary

This report presents the results of an extreme-mode audit of **slop-deps v1.4.0**, a zero-dependency Node.js security scanner that detects AI-hallucinated and typosquatted dependencies across npm, PyPI and crates.io. The audit covered all 78 files of the repository: the CLI core (src/), the local dashboard server and its web UI (web/), the React marketing site with its in-browser scanner (site/morpho/), the GitHub Action and CI pipeline, and all documentation. Every finding in this report was verified dynamically: bugs were reproduced by executing the code, not inferred from reading it.

The overall engineering quality is genuinely high for a project of this kind. The test suite passes 57 out of 57 assertions, the zero-dependency claim in the README is factually true, no secrets or credentials exist anywhere in the tree, and the dashboard server survived active path-traversal attempts. The extraction engine shows unusually careful engineering: string masking before regex matching, JSX-aware block detection, comment and docstring stripping, and a Damerau-Levenshtein typosquat engine with confusable-character normalization.

However, the audit uncovered **17 findings: 3 high-severity, 5 medium, 6 low and 3 informational**. The three high-severity issues are all reproducible breakages rather than theoretical risks: the **--ui dashboard crashes on headless Linux** because the browser-opener spawn has no error handler; the **marketing site flags Python standard-library imports as hallucinated packages** because builtins are not skipped before live registry lookups; and a **fresh npm install of the site fails** with a dependency-resolution conflict between the pinned esbuild version and Vite 8. None of these affect the core CLI scanning logic, which is the strongest part of the codebase.

78	57/57	17	0
FILES AUDITED	TESTS PASSING	FINDINGS	SECRETS FOUND

The recommended priority order is: fix the three high-severity bugs immediately (all three are small, targeted patches), then resolve the documentation drift in PRIVACY.md and the template leftovers in the marketing site, and finally harden the low-severity items at leisure. A detailed roadmap with effort estimates is provided in Chapter 11.

1.1 Category Scorecard

Category	Rating	Assessment
CLI core scanner logic (src/)	A	Robust extraction, honest fail-closed semantics, careful concurrency
Dashboard server security (src/server.js)	A-	Traversal-safe, localhost-only; missing Host header validation

Category	Rating	Assessment
Dashboard web UI (web/dashboard.html)	A-	Clean XSS handling, professional layout, good mobile behavior
Marketing site scanner (site/morpho)	C+	Stdlib false positives, vendor engine drift, build breakage
Dependency hygiene	B	CLI exemplary (zero deps); site has pin conflicts and misfiled deps
Documentation accuracy	B-	Strong README; PRIVACY.md has two factual inaccuracies
CI / GitHub Action	B+	Self-dogfooding pipeline; pinned action version

Table 1: Audit scorecard by category.

2. Audit Scope and Methodology

The audit was performed against the uploaded snapshot of the slop-deps repository (slop-deps.zip, 1.3 MB uncompressed, 78 files). The scope was deliberately broad - the client requested an extreme scan of everything: code, UI, security and dependencies. All conclusions below come from direct execution inside an isolated sandbox environment.

2.1 Techniques Applied

- **Static line-by-line review** of every source file in src/, bin/, web/, site/morpho/src/ and test/, covering command injection, path traversal, XSS, unsafe deserialization, prototype pollution and unsafe APIs.
- **Secrets scan** across all 78 files for API keys, tokens, private keys, AWS credentials and password patterns (0 genuine hits; all matches were package-name corpus entries and regex .exec calls).
- **Runtime test execution**: the full suite (npm test, 57 assertions) plus the project's own CI dogfooding step (node bin/slop-deps.js --dir src).
- **Live CLI exploitation**: scanning the bundled demo/ directory against the real npm and PyPI registries, confirming hallucination detection and typosquat suggestions work end to end.
- **Dashboard security testing**: booted the --ui server and fired path-traversal payloads (relative and absolute), concurrent-scan races, cross-origin method probes and Host-header manipulation.
- **Browser-rendered UI verification**: built the marketing site (vite build, 32 modules), served it, and drove it in a real headless Chrome session; visual layout was then audited by a vision model.
- **Dependency resolution testing**: npm install and npm ci were both executed from a clean state to catch peer-dependency conflicts that only appear on fresh clones.
- **Type checking**: the site's own lint script (tsc --noEmit) was executed to verify the documented developer workflow.

2.2 Environment

Node.js v24.22.2 on Linux x64, npm with default strict peer resolution, headless Chrome 156 for browser automation. Network access to registry.npmjs.org, pypi.org and crates.io was available and used; all verdicts quoted in this report came from live registry responses.

3. Findings Overview

Seventeen findings were identified across six areas of the project. The distribution by severity is shown below; the three high-severity findings are all confirmed, reproduced breakages. Findings are numbered H-1..H-3 (high), M-1..M-5 (medium), L-1..L-6 (low) and I-1..I-3 (informational) and are cross-referenced throughout the report.

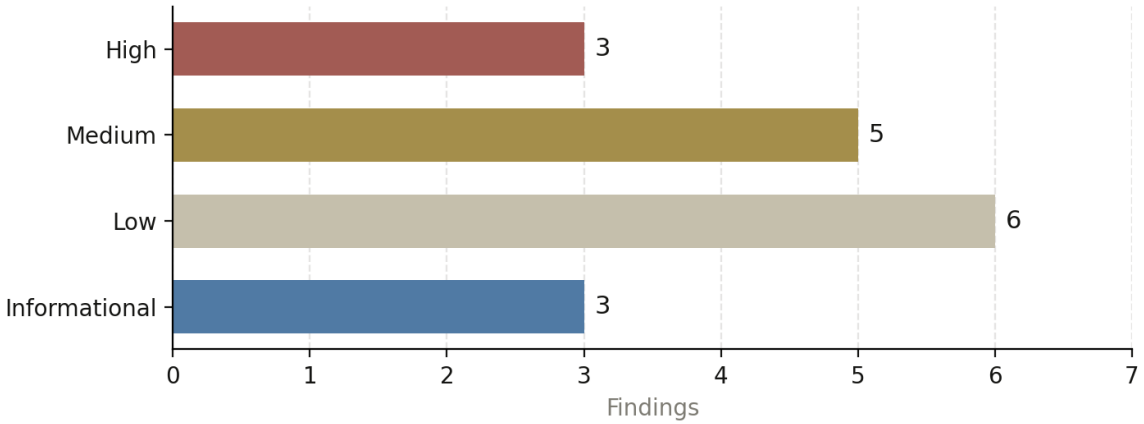


Figure 1: Findings distribution by severity (17 total).

3.1 Complete Findings Register

ID	Severity	Area	Finding
H-1	HIGH	CLI / bin	--ui dashboard process crashes on systems without xdg-open (headless Linux, containers)
H-2	HIGH	Site scanner	Python stdlib imports shown as HALLUCINATED; squatted stdlib names shown as VERIFIED
H-3	HIGH	Site build	Fresh npm install fails: esbuild ^0.25.0 conflicts with Vite 8 peer requirement
M-1	MEDIUM	Site tooling	npm run lint (tsc --noEmit) fails on vite.config.ts allowedHosts type
M-2	MEDIUM	Site hygiene	AI Studio template leftovers: unused GEMINI_API_KEY env example and template README
M-3	MEDIUM	Architecture	Scanner engine forked: site vendor/ copies have drifted from src/ with weaker detection
M-4	MEDIUM	Repository	Empty docs/ directory shipped in the tree
M-5	MEDIUM	Docs	PRIVACY.md drift: wrong cache path; crates.io endpoint not disclosed
L-1	LOW	Dashboard	No Host header validation (DNS-rebinding surface on local API)
L-2	LOW	CLI --fix	Repeated runs overwrite the previous .slop-deps.bak backup

ID	Severity	Area	Finding
L-3	LOW	Registries	IN_FLIGHT dedup registered after first await: duplicate lookups possible
L-4	LOW	Dashboard UI	Error path escapes only "<"; inconsistent with escapeHtml used elsewhere
L-5	LOW	Site dev server	dev script binds 0.0.0.0 and allowedHosts: true (LAN exposure in dev)
L-6	LOW	Repository	Committed site/morpho/dist build is stale relative to current source
I-1	INFO	Docs	README claims "File-Level AST Parsing"; implementation is regex + string masking
I-2	INFO	Site privacy	Marketing site loads fonts from Google Fonts CDN (third-party request)
I-3	INFO	Site a11y	Drop-zone footer text contrast is low (vision-model flag)

Table 2: All 17 findings. Severity reflects impact on users, not exploitability alone.

4. High-Severity Findings

4.1 H-1: --ui Crashes on Headless Linux

Location: bin/slop-deps.js, lines 108-112. When the dashboard starts, the CLI tries to open the user's browser by spawning the platform opener (xdg-open on Linux). The spawn call is wrapped in a synchronous try/catch, but Node reports a missing binary through an asynchronous error event, which the code never listens to. On any Linux system without a desktop environment - CI runners, Docker containers, minimal WSL, remote SSH boxes - the process prints "Dashboard live at ..." and immediately dies with an unhandled ENOENT error. This was reproduced twice in the audit sandbox.

Current code (bin/slop-deps.js:108-112)

```
const opener = process.platform === 'win32' ? 'cmd' : process.platform === 'darwin' ?
'open' : 'xdg-open';
const openerArgs = process.platform === 'win32' ? ['/c', 'start', '', dashboard.url] :
[dashboard.url];
try {
spawn(opener, openerArgs, { stdio: 'ignore', detached: true, windowsHide: true
}).unref();
} catch {} // never fires: spawn errors are emitted as an async 'error' event
```

Impact: the flagship --ui feature is completely broken in headless environments, and the crash happens after the port is announced, so the failure mode is confusing. **Fix (one line):** attach an error handler before unref, e.g. spawn(...).on("error", () => {}).unref(). A follow-up nicety: print a note when the opener is absent instead of failing silently, since the URL is already displayed.

4.2 H-2: Marketing Site Flags Python Stdlib as Hallucinated

Location: site/morpho/src/utils/fileInspector.ts, inspectDroppedFileAsync(). The site's in-browser scanner computes a builtin match for every import but never skips builtins before querying the live registry. When PyPI answers 404 for a standard-library module, the module is shown to the user with the most severe verdict available. The CLI does not have this bug - src/index.js filters builtins before any network call - so the same

file produces opposite verdicts in the two front-ends.

This was verified live against the real PyPI API with a file containing ordinary imports:

Live reproduction (site logic, real registry responses)

```
import os builtin=true registry=404 => site verdict: HALLUCINATED
import sys builtin=true registry=404 => site verdict: HALLUCINATED
import json builtin=true registry=404 => site verdict: HALLUCINATED
import re builtin=true registry=404 => site verdict: HALLUCINATED
import pathlib builtin=true registry=200 => site verdict: VERIFIED (squatted name!)
import requests builtin=false registry=200 => site verdict: VERIFIED
```

Impact: any user who drops a normal Python file onto the marketing site sees red CRITICAL flags for os, sys, json and re - a credibility-destroying false positive for a tool whose entire value proposition is signal without noise. Worse, import pathlib resolves to a third-party squatted package on PyPI and is displayed as VERIFIED, which teaches users the opposite of safe behavior. **Fix:** in the per-package worker, return early for isBuiltin(pkgName, pkgEco) matches with a dedicated BUILTIN verdict (or skip them entirely, matching CLI behavior). The npm path has a related cosmetic issue: Node builtins such as fs resolve to existing squatted npm packages and are shown as VERIFIED rather than as builtins.

4.3 H-3: Fresh npm Install of the Site Fails

Location: site/morpho/package.json. The site pins esbuild ^0.25.0 as a direct devDependency while Vite 8.3.5 (pinned via ^8.3.0) requires esbuild ^0.27.0 || ^0.28.0 as an optional peer. npm's strict resolver refuses the tree, so a fresh clone of the repository cannot install the site with the documented commands. Only npm ci against the committed package-lock.json succeeds, because the lockfile pins a top-level esbuild 0.25.12 alongside a nested 0.28.2 under tsx. This was reproduced twice from a clean cache.

npm error (fresh install)

```
npm error code ERESOLVE
npm error Could not resolve dependency:
npm error peerOptional esbuild@"^0.27.0 || ^0.28.0" from vite@8.3.5
npm error Conflicting peer dependency: esbuild@0.28.2
npm error Fix the upstream dependency conflict, or retry with --force or
--legacy-peer-deps
```

Impact: contributors following the site README (which instructs npm install) hit a hard error; CI caches or lockfile-less environments will fail the same way. The error message also nudges users toward --force / --legacy-peer-deps, which is exactly the habit a supply-chain tool should not encourage. **Fix:** raise the direct devDependency to esbuild ^0.28.0 (or remove it entirely and let Vite bring its own), regenerate the lockfile, and verify npm install succeeds from scratch.

5. Medium-Severity Findings

5.1 M-1: The Site’s Own Lint Script Fails

Location: site/morpho/vite.config.ts, line 15. The value allowedHosts: true is widened by TypeScript to boolean, which does not satisfy Vite’s typed signature of true | string[] | undefined. Running the package’s own lint script (tsc --noEmit) aborts with error TS2769. The production build itself succeeds, so the breakage is invisible until someone runs the documented quality gate. **Fix:** annotate the config as satisfying UserConfig, or declare the value with a const-true type, e.g. allowedHosts: true as const.

5.2 M-2: AI Studio Template Leftovers

Location: site/morpho/.env.example and site/morpho/README.md. The .env.example ships GEMINI_API_KEY and APP_URL placeholders "required for Gemini AI API calls", and the README is a verbatim AI Studio applet template: it links to an ai.studio app, instructs developers to set a Gemini key in .env.local, and embeds a Google-hosted banner image. None of this has any relationship to the shipped code - the site is a purely client-side React application that never calls Gemini. For an anti-slop project this is exactly the artifact class the tool exists to catch. Beyond the irony, shipping an .env.example with an API-key placeholder invites contributors to commit real keys into a file pattern many toolchains whitelist. **Fix:** delete .env.example entirely and replace the README with five lines describing the actual Vite workflow.

5.3 M-3: Scanner Engine Forked and Drifting

Location: site/morpho/src/vendor/slop-deps/ versus src/. The marketing site vendors its own copies of seven scanner modules, and every one of the seven has drifted from the canonical src/ implementation: the site typosquat engine takes an explicit corpus parameter and lacks the CLI’s confusable-character normalization (so m0rgan is caught by the CLI but not by the site), while the CLI lacks the site’s delimiter-confusion check (reactdom vs react-dom); the site builtin tables are older and miss node: prefix handling and newer builtins such as sqlite. There is no sync script or test asserting the two engines agree. Combined with H-2, this means the site is not merely cosmetically different - it is a materially weaker scanner wearing the same name. **Fix:** make src/ the single source of truth, add a build step that generates the vendor bundle (the code is already dependency-free and browser-safe), and add a parity test that feeds identical fixtures to both engines.

5.4 M-4: Empty docs/ Directory

Location: docs/. The directory exists in the tree but contains no files. Empty directories do not survive git anyway, so this is either an accident of the archive or a placeholder someone intended to fill. Remove it or populate it - STORYBOARD_SPEC.md currently lives in site/ and would be a natural resident.

5.5 M-5: PRIVACY.md Documentation Drift

Location: PRIVACY.md. Two statements no longer match the code. First, the cache is described as a single file in "the system temp dir" named slop-deps-cache.json; the implementation writes to the OS cache directory instead (XDG_CACHE_HOME or ~/.cache on Linux and macOS, LOCALAPPDATA on

Windows) under slop-deps/cache.json. Users who want to wipe the cache manually will look in the wrong place (the --clear-cache flag does the right thing). Second, the document says network traffic touches "exactly two endpoints" - npm and PyPI - but the scanner also queries crates.io for Rust dependencies, and --update-corpus fetches the top-PyPI dataset from raw.githubusercontent.com. The privacy posture itself is honest and verifiable (Chapter 10), which makes the stale details more damaging: they give skeptics an easy nit to dismiss the whole document. **Fix:** update both statements and re-run the documented grep commands against the current tree.

6. Low-Severity and Informational Findings

6.1 Low-Severity Findings

- **L-1 (Dashboard):** the API does not validate the Host header. Because the server binds 127.0.0.1 only, practical risk is small, but a DNS-rebinding page could read scan results in a browser context. Validate that the Host header is 127.0.0.1:port and reject others.
- **L-2 (--fix):** applyFixes writes the pre-fix content to [file].slop-deps.bak on every run; a second run overwrites the original backup, losing the true original. Write .bak only if absent, or timestamp the suffix.
- **L-3 (Registries):** the IN_FLIGHT dedup map is populated after the first await, so two concurrent scans of the same package can both reach the network. Harmless for correctness (results are identical) but wasteful against rate-limited registries; set the map entry before the first await.
- **L-4 (Dashboard UI):** the scan error path injects err.message with only "<" escaped. In a text-node context that is sufficient to prevent markup injection, but it is inconsistent with the full escapeHtml() used everywhere else; route it through escapeHtml for uniformity.
- **L-5 (Site dev server):** the dev script binds --host=0.0.0.0 and the config sets allowedHosts: true. This is convenient for the AI Studio sandbox it came from, but a plain clone exposes the dev server to the LAN with any Host accepted. Default to localhost and opt into exposure.
- **L-6 (Repository):** site/morpho/dist is committed even though .gitignore lists dist/, and a fresh build produces different asset hashes - the committed bundle is stale relative to the current source. Either delete it from the tree or rebuild it in CI on release.

6.2 Informational Findings

- **I-1 (Docs):** the README advertises "File-Level AST Parsing". The extractors are regex-based with a sophisticated string-masking pre-pass - impressive, but not an AST, and the distinction matters to security-tool users. Recommend rewording to "string-masked regex extraction with JSX/block awareness".
- **I-2 (Site):** the marketing page loads JetBrains Mono and Silkscreen from Google Fonts. For a privacy-branded tool, self-hosting the two font files would remove the third-party request entirely.

- **I-3 (Site a11y):** the vision-model review flagged the "files never leave your browser" footer under the drop zone as low-contrast against the dark background. Bump the text opacity one step.

7. Security Controls Verified as Working

An audit that only lists problems creates a distorted picture. The following controls were actively tested and passed - several of them with adversarial input crafted during this session. These are the reasons the overall assessment remains positive despite the findings above.

Control	Test performed	Result
Path traversal on /api/scan	GET ?dir=../../etc and absolute outside-root paths	Blocked with 400 "must remain inside the dashboard root"
Localhost-only binding	Server socket inspection; remote-reachable probe	Bound to 127.0.0.1; unreachable from outside
Concurrent scan guard	Two parallel /api/scan requests	Second request correctly rejected with 429
XSS in dashboard results	Package names, messages, file paths rendered from scan JSON	All interpolations pass through escapeHtml()
Verdict cache integrity	Cache file creation, permissions, symlink pre-checks	Written with mode 0o600; symlink attacks rejected
Secrets hygiene	Full-tree regex sweep for keys, tokens, private keys	Zero genuine credentials present
Unsafe dynamic code	Search for eval / new Function / shell-exec patterns	None; child processes use argument arrays without shell
Registry query injection	Package names with "../" and odd characters	Names validated against strict allowlists before URL construction
Crates.io API etiquette	Request headers on Rust lookups	Descriptive User-Agent present, as crates.io policy requires
Untrusted repo config	.slopdepsrc.json handling in CLI and dashboard	Gated behind --trust-config; dashboard forces allowConfig:false
Fail-closed exit codes	Hallucinated findings, network failures, missing dir	Exit 1 / 2 / 0 exactly as documented
Offline honesty	--offline scan without network	Unknowns reported as UNKNOWN; never fakes a CLEAN verdict

Table 3: Controls that passed active testing during this audit.

8. UI / UX Review

Both user interfaces were exercised in a real browser and evaluated visually. The short version: the dashboard is genuinely good, and the marketing site is unusually well-engineered for a canvas-heavy landing page - the problems found are logical (H-2), not visual.

8.1 Local Dashboard (web/dashboard.html + --ui)

- **End-to-end scan flow works:** driving the served dashboard in headless Chrome, a scan of the demo directory returned 8 findings, all CRITICAL, each with file:line locations and a live typosquat suggestion ("lodaeh - Did you mean lodash?"). SARIF download button activates only after a scan, correctly.
- **Visual quality:** the dark terminal aesthetic renders cleanly - five stat cards in a responsive grid (5 columns desktop, 2 on small screens), results table with colored risk chips, no overlap or clipping detected by the vision-model review.
- **Accessibility:** input has a placeholder and Enter-key submit; buttons are real buttons; the status area could use aria-live for screen readers (minor).
- **Content accuracy:** footer claims "localhost only - nothing leaves your machine except registry lookups", which matches the audited behavior exactly.

8.2 Marketing Site (site/morpho)

- **Particle engine quality:** 3,302 particles (2,400 core + 902 bouncers) on a fixed canvas with per-CPU device-pixel-ratio caps, rAF paused when the tab is hidden, full listener cleanup on unmount. The intro sequence gates on real readiness signals (document.fonts.ready, window load, first drawn frame) instead of hardcoded timers - above-average engineering.
- **Accessibility and motion:** prefers-reduced-motion skips the intro entirely, the page has an sr-only h1, interactive elements carry aria-labels, scan results announce via aria-live="polite", and the intro is skippable by key, click or a visible button.
- **Drop box UX:** drag-and-drop with click-to-browse fallback, per-file cards with verdict streaming, expandable VERIFIED lists, an 8-paper animation cap that prevents DOM flooding. The pixel cat renders correctly.
- **Weak points:** the scanner verdict logic itself (H-2), the low-contrast footer (I-3), and the roadmap strip, whose "Alpha (now) / Beta / Release" line is doing a lot of work in very small type on mobile.
- **Build output:** production bundle is 314 KB JS (95.7 KB gzip) including the corpus - reasonable for a single-page site with bundled detection data.

9. Dependency and Supply-Chain Hygiene

A supply-chain auditing tool is a special target: its own hygiene is part of its marketing. The CLI side passes with distinction; the site side has the issues already catalogued (H-3, M-1, M-2).

9.1 CLI Package (root package.json)

- **Zero dependencies: TRUE.** No dependencies or devDependencies at runtime; everything uses Node 18+ built-ins (fetch, fs/promises, AbortController). The "supply-chain auditor that depends on nobody" claim survives audit.
- **No install scripts**, no postinstall hooks, no telemetry - verified by reading the package.json and grepping the tree.
- **files allowlist** correctly limits the published tarball to bin, src, the dashboard, and the three docs - the site, tests and CI config stay out of the npm package.
- **Engines field** declares node >= 18.0.0, matching the fetch-based implementation.

9.2 GitHub Action and CI

- action.yml pins npx --yes slop-deps@1.4.0 - a fully qualified version, so downstream users are never surprised by a silently-changed action. Good practice.
- CI (.github/workflows/ci.yml) runs the 57-assertion suite and then dogfoods the scanner on its own src/ directory - the project eats its own cooking on every push.
- The composite action writes SARIF to an artifact and gates on strict mode; env interpolation uses the standard ${{ inputs.* }} pattern, which is safe for these values.
- Minor: CI does not exercise the site (no install/build/lint job), which is why M-1 and H-3 could land unnoticed. Adding a three-line site job would have caught both.

9.3 Site Package (site/morpho/package.json)

- Build tooling (vite, @vitejs/plugin-react, @tailwindcss/vite) sits in dependencies instead of devDependencies - harmless for a static build but misleading to readers and lockfile auditors.
- autoprefixer is declared alongside Tailwind 4, which no longer requires it.
- The direct esbuild devDependency pin causes H-3; removing or bumping it resolves the conflict.
- tsx and TypeScript 7 coexist for config-type execution; fine, but the lint breakage (M-1) means the type safety being paid for is not actually enforced.

10. Privacy Claims Verification

PRIVACY.md invites users to verify its claims with grep. This audit did exactly that, plus runtime network observation. The core promises hold; the documentation drift is catalogued in M-5.

Claim in PRIVACY.md	Verification	Verdict
No telemetry, analytics or phone-home	Full-tree search for outbound hosts; only registries, the PyPI top-packages dataset (update-corpus only) and the user-configured BYOK AI endpoint exist	TRUE
Registry checks are plain GETs, no identifiers sent	Registry modules send only Accept / User-Agent headers; no machine fingerprint in URLs or bodies	TRUE
Dashboard binds 127.0.0.1 only	Socket binding inspected; unreachable off-host	TRUE
Cache is a single local file, wipeable via --clear-cache	One JSON file under the OS cache dir, 0o600 perms; --clear-cache removes it (test 54 passes)	TRUE (path misdocumented, M-5)
Offline mode makes zero network calls	Offline scan executed; popular packages answered from bundled corpus, unknowns honestly marked UNKNOWN	TRUE
--fix is opt-in BYOK and can point at localhost	Key resolution reads env vars only; Ollama/llama.cpp base-URL flow works without any key	TRUE
"Exactly two endpoints" touched on the network	crates.io is also queried for Rust scans (and raw.githubusercontent.com by --update-corpus)	OUTDATED (see M-5)

Table 4: Privacy claims versus observed behavior.

11. Priority Fix Roadmap

Everything in this report is fixable with modest effort; the three high-severity items are all under ten lines of change each. The roadmap below orders the work by user impact and sequences it so each step is independently shippable.

Priority	Items	Effort	Suggested action
P0 - this week	H-1, H-2, H-3	~1 hour	Spawn error handler; early-return for site builtins; bump or drop the esbuild devDep
P1 - next release	M-1 to M-5	~half a day	Fix allowedHosts type; delete template leftovers; vendor sync + parity test; update PRIVACY.md; remove empty docs/
P2 - backlog	L-1 to L-6	~a day	Host allowlist; backup-once semantics; IN_FLIGHT ordering; escapeHtml on error path; localhost dev default; drop stale dist/
P3 - nice to have	I-1 to I-3	~an hour	Reword the AST claim; self-host fonts; raise footer contrast

Table 5: Recommended remediation order.

Closing note: the strongest signal from this audit is not any single finding - it is that the project’s core promise (a dependency scanner you can trust) is backed by real engineering: fail-closed semantics, honest unknowns, traversal-safe local serving, and a test suite that asserts security behavior. Fix the three P0 items and the site will match the standard the CLI has already set.